

Collections

Advanced C# Programming - NET .022

Prepared By: Rupinder Sandhu
Rupinder.sandhu@humber.ca



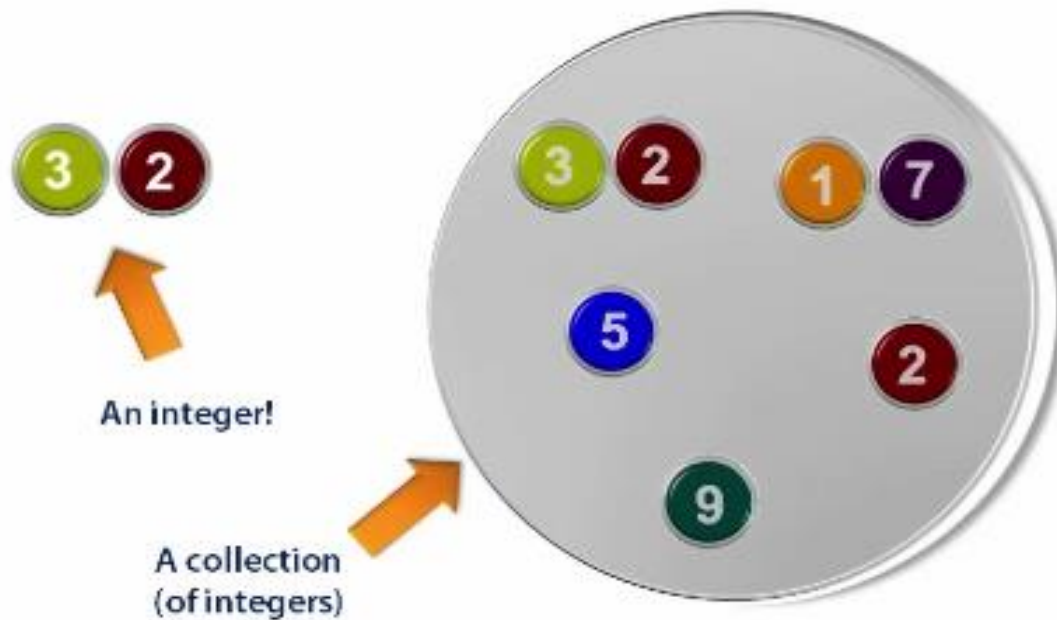
**WE ARE
HUMBER**

Agenda

- Understanding collection interfaces and types
- Working with lists, queues, and stacks
- Working with linked and sorted lists
- Using dictionaries and sets

What is a Collection?

- A type responsible for managing a collection of objects in memory



There are two ways to group objects

- **Arrays**
 - Arrays are most useful for creating and working with a fixed number of strongly typed objects.
- **Collection**
 - Collections provide a more flexible way to work with groups of objects. The group can grow and shrink dynamically as the need arises.

There are many collection types which fall into 3 specific groups

- Lists
- Dictionaries
- Sets

[Comparing Arrays and Collections](#)

Lists

- A list of elements, efficient for accessing elements, good memory usage
- Items can be looked up via their index
 - Indexes by default begin at position 0 not 1 – zero based indexing
- Types
 - Arrays T[]
 - List<T>
 - Collection<T>
 - ReadOnlyCollection<T>
 - ObservableCollection<T>
- Interface
 - IList<T> - Contract for indexed based lists

Dictionaries

- Useful when identifying elements by something that is unique to that element (a key)
- Underlying type is a Hash Table
- Types
 - Dictionary<Tkey, Tvalue>
- Interfaces
 - IDictionary<Tkey, Tvalue>

List vs. Dictionary Performance

Looking up an element



Sets

- Sets treat the collection as a group not on grouping individual items
- Intersections
 - The collection of elements that are in 2 or more sets
- Unions
 - The collection of elements that are in any of several sets
- Sets are collections that support being combined to form new sets
- Types
 - `HashSet<T>`
- Interfaces
 - `ISet<T>`

Similarities between Dictionaries and Sets



Collection Operations

- Reading
 - Lookup a single element (key or index) –
 - Most collections allow it
 - Sets, stacks and queues do not allow looking up individual items
 - Enumerate all items in a collection (foreach loop)
 - Lists – enumerate in index order
 - Dictionaries – do not maintain an order

Collection Operations (cont.)

- Writing
 - Adding
 - Inserting
 - Replacing
 - Removing

Arrays

- An array is an index based list
- Rich API
 - Most collection structures are based on 1 or more arrays
- Very Lightweight
 - Implemented inside the CLR itself
- Special C# Syntax
 - Square brackets '['']'
- Fixed Size
 - Cannot add or remove from arrays

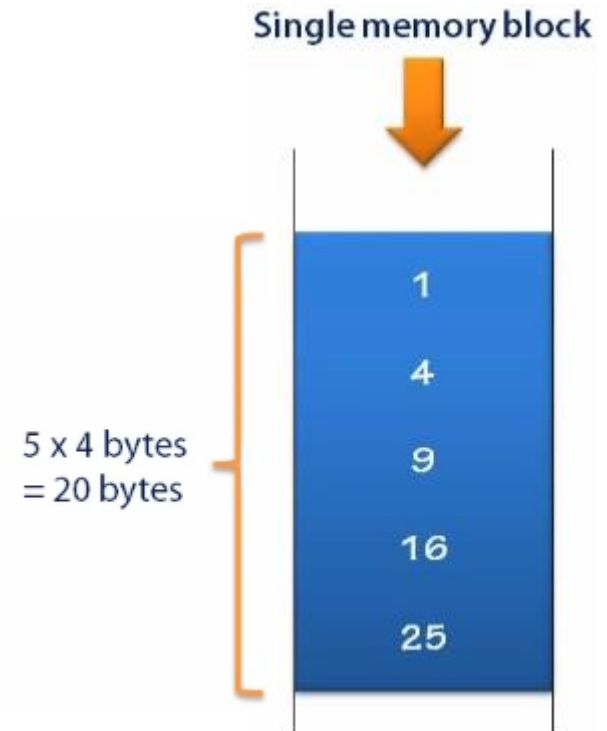
<https://dotnetfiddle.net/ZH2tIm>

How do Arrays work?

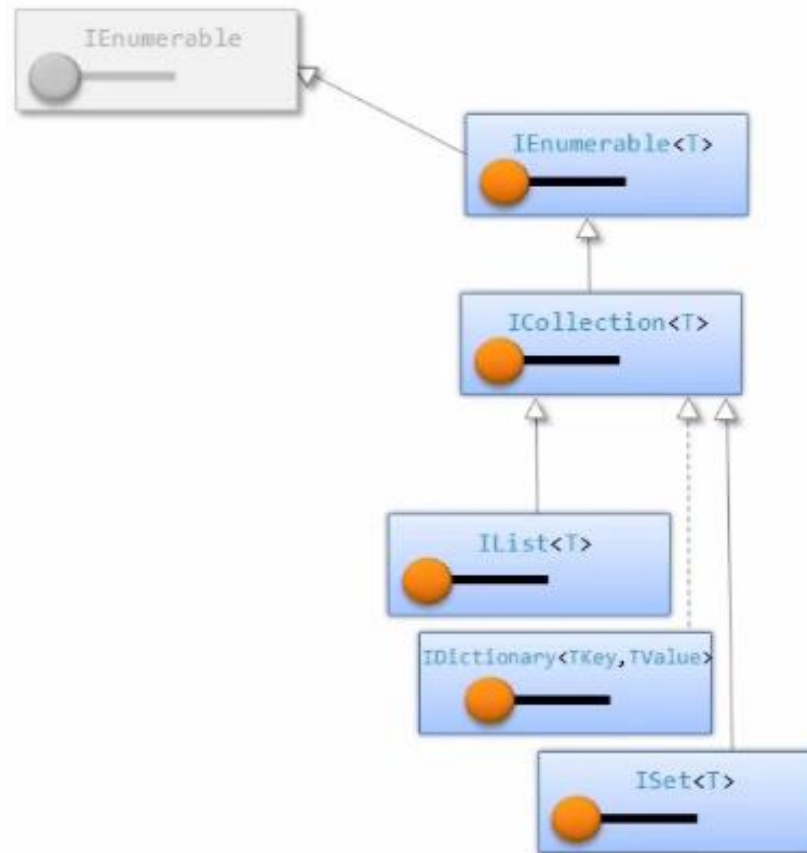
Array of 5 integers:

```
int[] squares = {1, 4, 9, 16, 25};
```

- Elements go one after the other in a block of memory
- One int requires 4 bytes of memory
- Allocated memory =
 - N elements of S bytes
- If you know the index of the item you want it is easy to work out it's position in memory
 - $\text{Address} = X + (n - 1) * S$



Collection Hierarchy



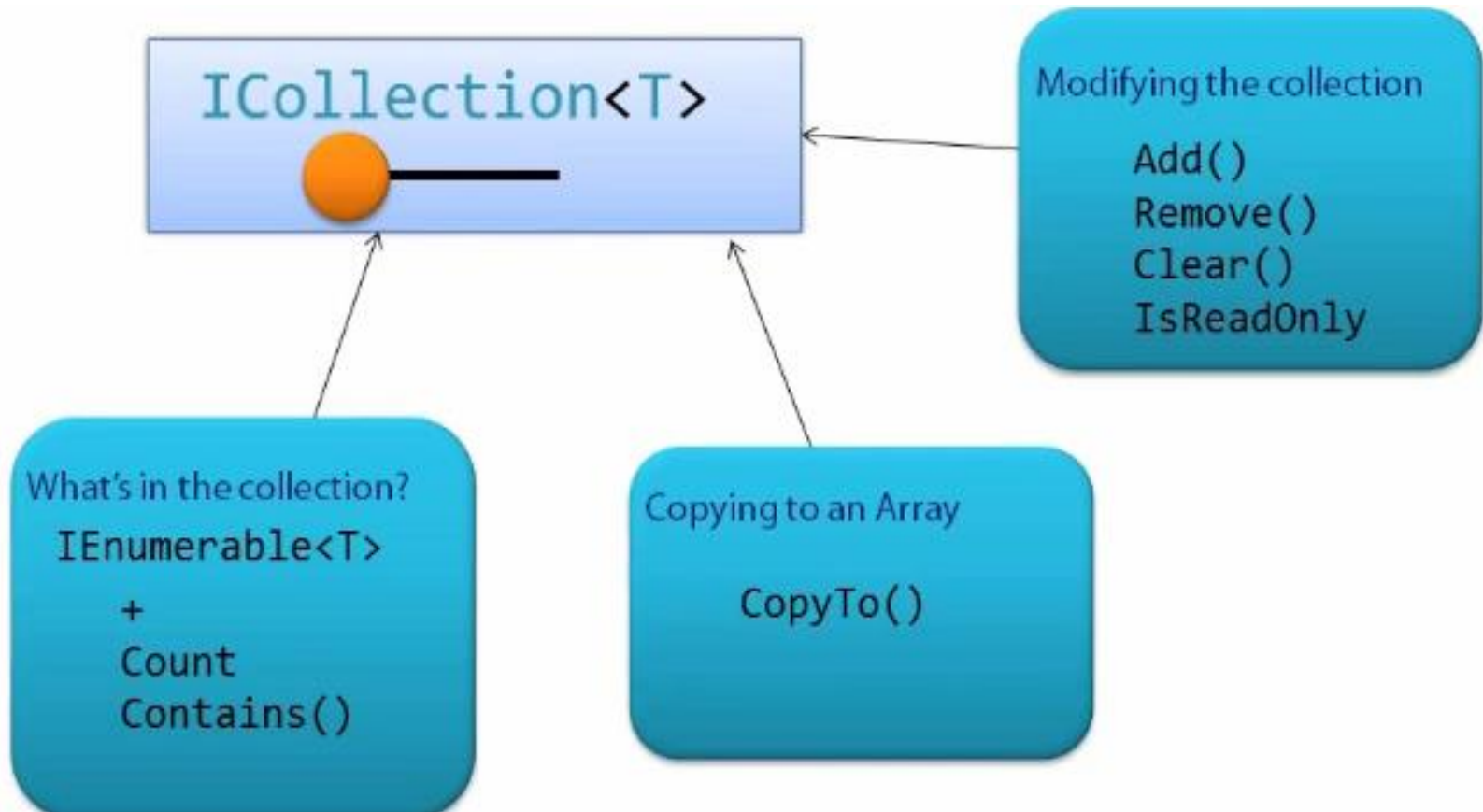
Collection Interfaces

Interface	Descriptions
IEnumerable<T>	• Allows iteration over each item of a collection • Defines the method GetEnumerator
ICollection<T>	• Adds Count property, and implements copy the collection to an array (CopyTo). • Add and remove items from the collection (Add, Remove, Clear)
IList<T>	• Elements can be accessed from their position. • This interface defines an indexer, as well as ways to insert or remove items from specific positions (Insert, RemoveAt)
IDictionary<T>	• Generic collection classes that have a key and a value.
ISet<T>	• Allows intersection and union calculations across collections

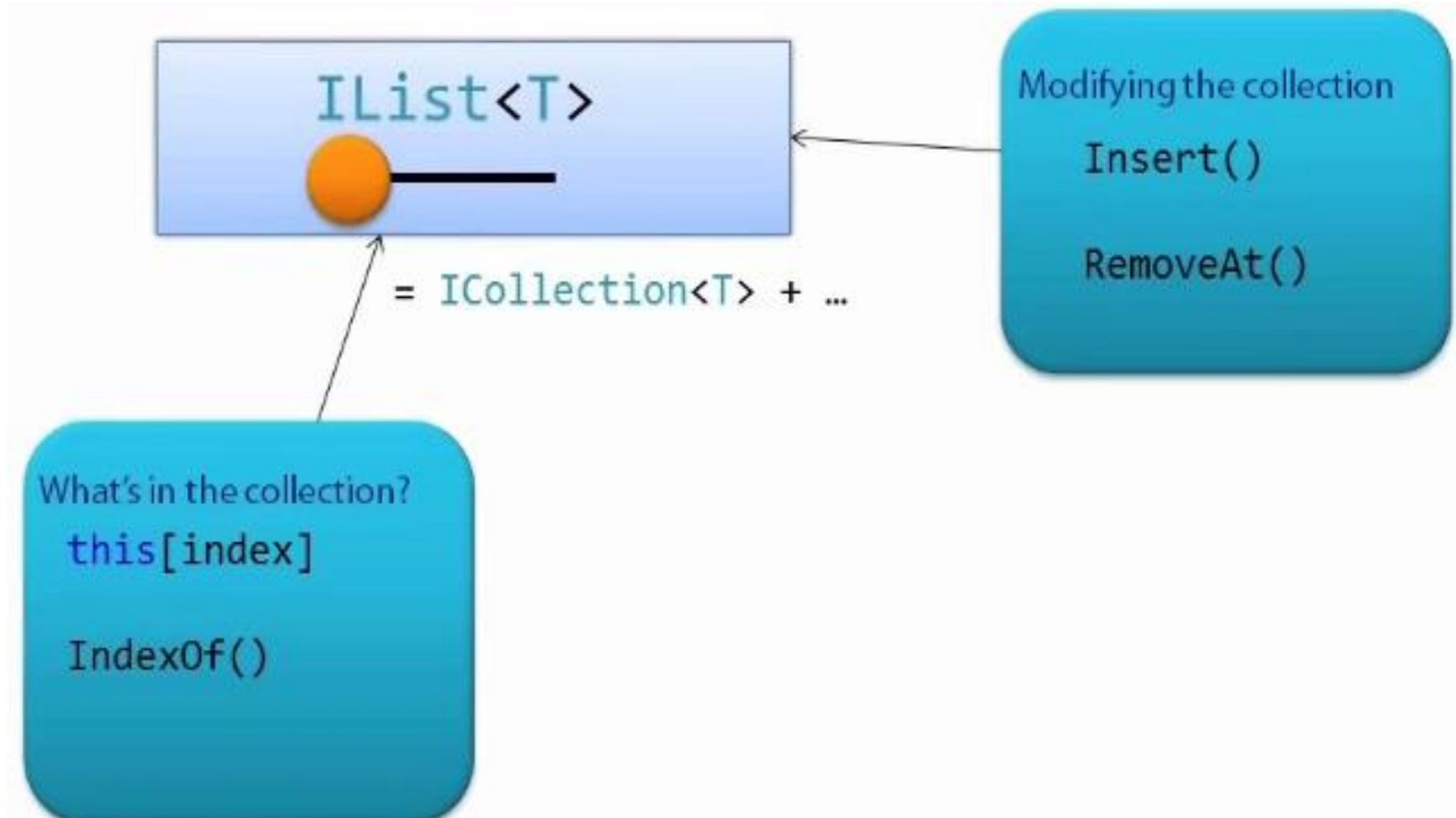
IEnumerable<T>

- 1 method – GetEnumerator()
- The collection is 'Enumerable'
- GetEnumerator returns the Enumerator (the thing that does the enumerating)
- ForEach works for all collections (anything that implements IEnumerable<T>)

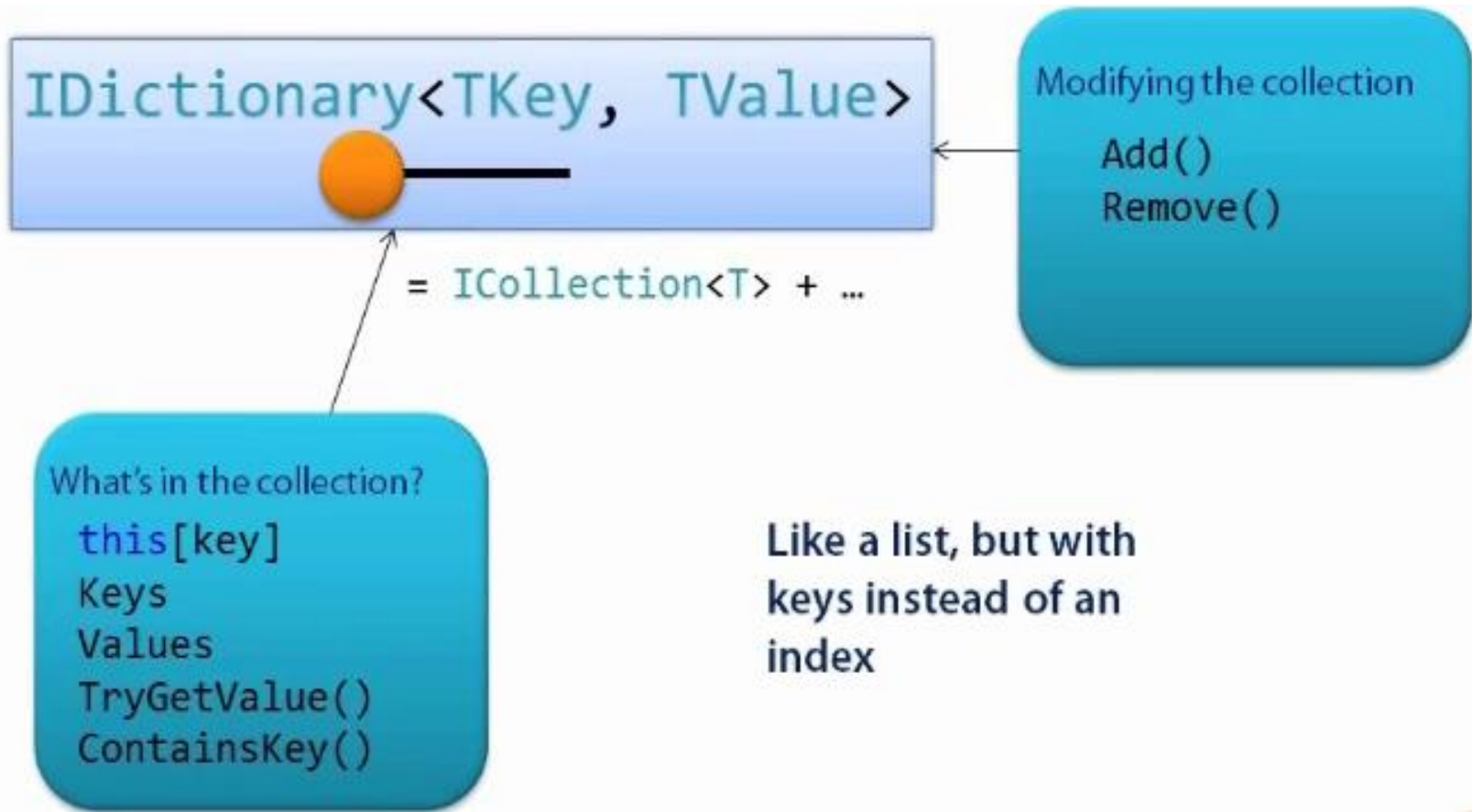
ICollection<T>



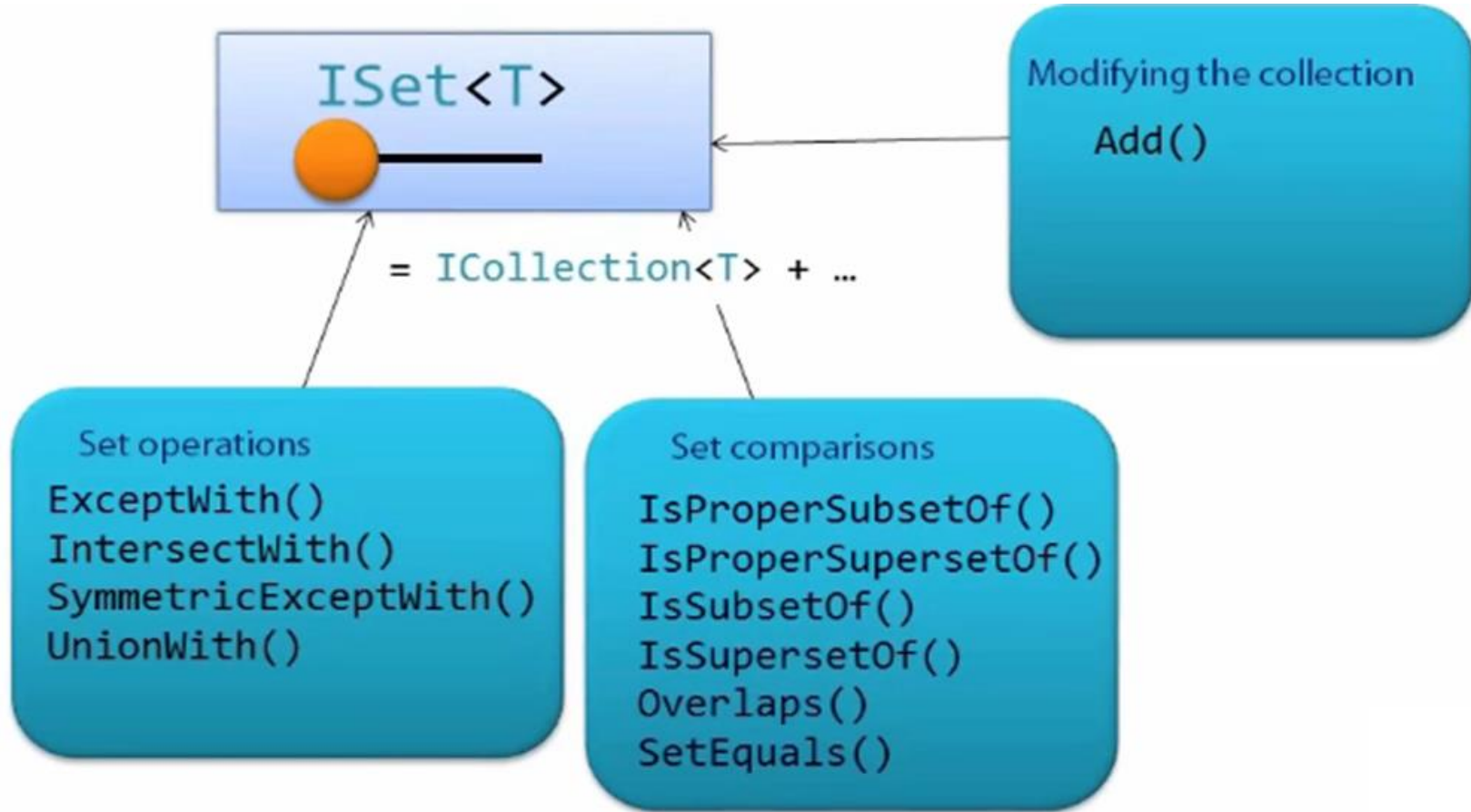
IList<T>



IDictionary<Tkey, TValue>



ISet<T>



Collection Implementations

- Lists
- Linked Lists
- Stacks
- Queues
- Dictionaries
- Sets
- ... and many more

Lists

- List<T>
 - Extensive API
 - Acts like an array but allows adding and removing elements
 - Probably the most widely used collection

<https://dotnetfiddle.net/loHnQW>

Collection

- Collection<T>
 - Provides an implementation of IList<T>
 - Like a List<T> but allows customization
 - E.g. A list that must only accept non-whitespace, non-null strings

<https://dotnetfiddle.net/9jwayO>

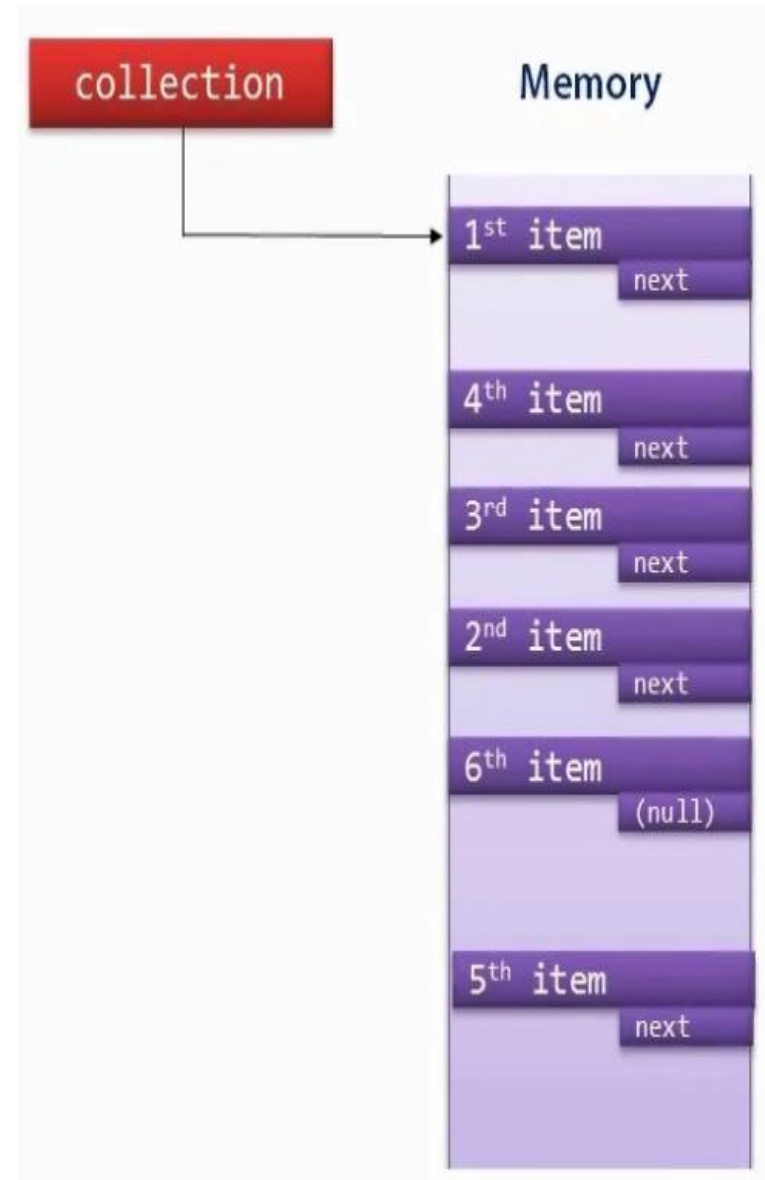
ObservableCollection

- ObservableCollection<T>
- List that provides change notifications

<https://dotnetfiddle.net/dcG72B>

Link Lists

- `LinkedList<T>`
- Collection that's quick at adding and removing items
- Not based on an array
 - Does not have arrays issue of a single block of memory
- Single linked list or double linked list
- No index-based access to elements
- Uses more memory than an array



Stack

- Stack<T>
- Last-in first-out
- Push
 - Adds an item to the stack
- Pop
 - Pops an item off the stack
- Peek
 - Peek allows you to see what item is at the top of the stack

<https://dotnetfiddle.net/g7SURq>

Queue

- Queue<T>
- First-in first-out
 - Opposite of the stack
- Enqueue
 - Adds an item to the queue
- Dequeue
 - De-queues an item off the queue
- Peek
 - Peek allows you to see what item is at the front of the queue

Dictionaries

- Dictionary<TKey, TValue>
- Enables you to access an element based on a key
- A dictionary maintains a set of 'buckets' that it uses to allocate keys
- Based on a hash table

<https://dotnetfiddle.net/22mCfA>

Sets

- HashSet<T>
- Elements have no order
- No keys
- Just element values .. Nothing else
- No way to lookup any individual value
- It does guarantee uniqueness

<https://dotnetfiddle.net/g7SURq>

THANK YOU.



WE ARE

HUMBER